



Lab 11 – Worksheet

Name: Arwa Ali Asghar, Areesha Noman, Emaan Fatima, Arsheel Abbas & Abdul Ghani	ID:	Section:T3
---	-----	------------

Task a: vga_sync

Code: Design module **vga_sync**

Provide appropriately commented code for designed module, the code should contain meaningful variable naming.

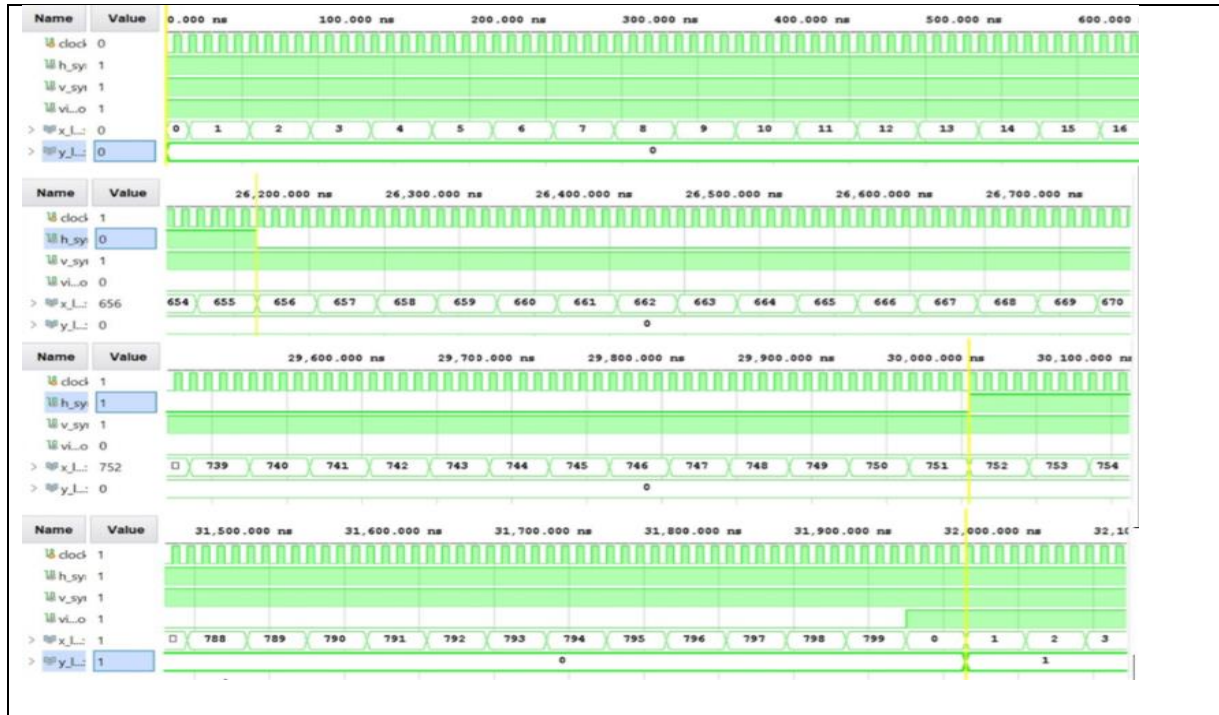
**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

```
module VGA_sync(  
    input [9:0] h_count,  
    input [9:0] v_count,  
    output h_sync,  
    output v_sync,  
    output video_on,  
    output [9:0] x_loc,  
    output [9:0] y_loc  
);  
  
localparam HD = 640;    // Horizontal Display Area  
localparam HF = 16;    // Horizontal (Front Porch) Right Border  
localparam HR = 96;    // Horizontal Retrace  
localparam HB = 48;    // Horizontal (Back Porch) Left Border  
  
localparam VD = 480;    // Vertical Display Area  
localparam VF = 10;    // Vertical (Front Porch) BottomBorder  
localparam VR = 2;     // Vertical Retrace  
localparam VB = 33;    // Vertical (Back Porch)Top Border  
  
assign h_sync = (h_count < HD+HF) || (h_count >= HD+HR+HF)?1:0;  
assign v_sync = (v_count < VD+VF) || (v_count >= VD+VR+VF)?1:0;  
assign x_loc = h_count;  
assign y_loc = v_count;  
assign video_on = (h_count < HD) && (v_count < VD)?1:0;  
endmodule
```



Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped. make sure to scale properly for visibility of all case.*



Comments

**Observation/Comments on the obtained results/working of code.*

This output matches the expected output.

Task b: Pixel Generation

Code: Design pixel_gen

Provide appropriately commented code for designed module and the testbench, the code should contain meaningful variable naming.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

```
module pixel_gen(  
    input clk_d,  
    input [9:0] pixel_x,  
    input [9:0] pixel_y,  
    input video_on,  
    output reg [3:0] red=0,  
    output reg [3:0] green=0,  
    output reg [3:0] blue=0  
);  
    always @(posedge clk_d)  
        begin  
            if ((pixel_x ==0) || (pixel_x ==639) || (pixel_y ==0) || (pixel_y==479))  
                begin  
                    red <= 4'hF; green <= 4'hF; blue <= 4'hF;  
                end  
            else  
                begin  
                    red <= video_on?(pixel_y>240? 4'hF:4'h0):(4'h0);  
                    green <= video_on?(pixel_y >240?4'h0:4'hF):(4'h0);  
                    blue <= 4'h0;  
                end  
            end  
        end  
endmodule
```

Task c: Top-Level Module/VGA Controller

Create a Top-Level Module as in Figure 11. 7. This module will receive a 100Mhz Clock as input and generate 1-bit *h_sync* and *v_sync* signals and three 4-bit signals for Red, Green and Blue color.

Code: Design module & testbench

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable names.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

```
module Top_level_lab11_task_c1
    input clock,
    output h_sync,
    output v_sync,
    output [3:0] red,
    output [3:0] green,
    output [3:0] blue
);
    wire clk_div;
    wire video_on;
    wire [9:0] h_count;
    wire [9:0] v_count;
    wire [9:0] x_loc;
    wire [9:0] y_loc;

    clock_divider a0(.clk(clock), .clk_div(clk_div));
    h_counter a1(.clk(clk_div), .h_count(h_count), .trig_v(trig_v));
    v_counter a3(.clk(clk_div), .enable_v(trig_v), .v_count(v_count));
    VGA_sync a2(.h_count(h_count), .v_count(v_count), .h_sync(h_sync), .v_sync(v_sync), .video_on(video_on), .x_loc(x_loc), .y_loc(y_loc));
    pixel_gen p1(.clk_d(clk_div), .pixel_x(x_loc), .pixel_y(y_loc), .video_on(video_on), .red(red), .green(green), .blue(blue));
endmodule
```

Constraint File

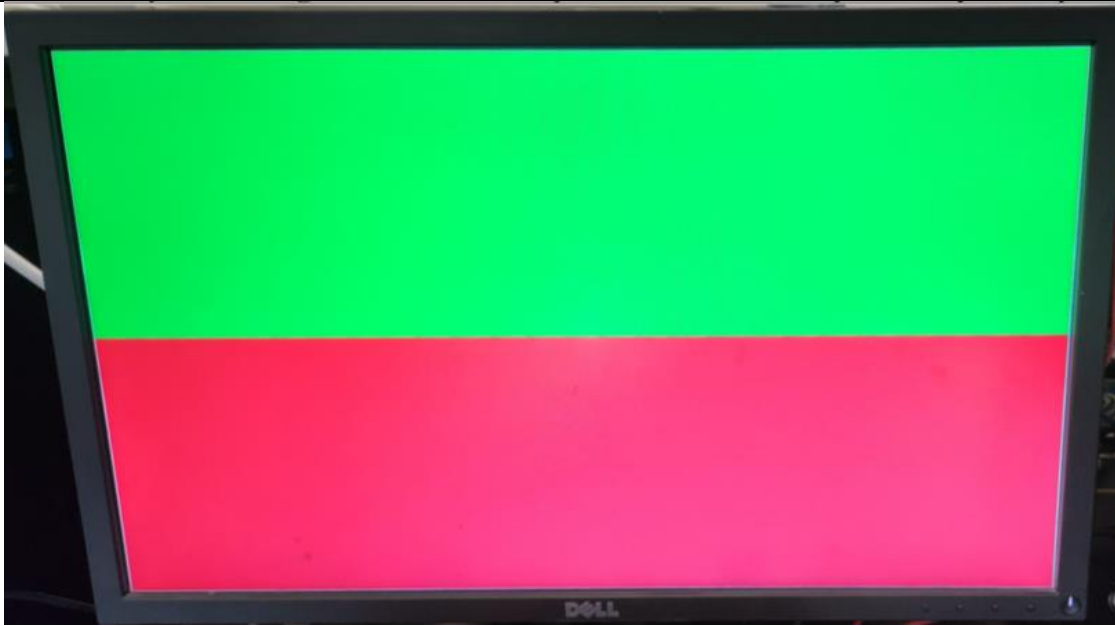
Copy & paste your constraint file (.xdc) here:

```
set_property IOSTANDARD LVCMOS33 [get_ports {blue[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {blue[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {blue[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {blue[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {green[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {green[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {green[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {green[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {red[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {red[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {red[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {red[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports clock]
set_property IOSTANDARD LVCMOS33 [get_ports h_sync]
set_property IOSTANDARD LVCMOS33 [get_ports v_sync]
set_property PACKAGE_PIN W5 [get_ports clock]
set_property PACKAGE_PIN P19 [get_ports h_sync]
set_property PACKAGE_PIN R19 [get_ports v_sync]
set_property PACKAGE_PIN G19 [get_ports {red[0]}]
set_property PACKAGE_PIN H19 [get_ports {red[1]}]
set_property PACKAGE_PIN J19 [get_ports {red[2]}]
set_property PACKAGE_PIN N19 [get_ports {red[3]}]
set_property PACKAGE_PIN J17 [get_ports {green[0]}]
set_property PACKAGE_PIN H17 [get_ports {green[1]}]
set_property PACKAGE_PIN G17 [get_ports {green[2]}]
set_property PACKAGE_PIN D17 [get_ports {green[3]}]
set_property PACKAGE_PIN N18 [get_ports {blue[0]}]
set_property PACKAGE_PIN L18 [get_ports {blue[1]}]
set_property PACKAGE_PIN K18 [get_ports {blue[2]}]
set_property PACKAGE_PIN J18 [get_ports {blue[3]}]
```

Results

Implement the VGA controller on BASYS-03 board and show output on VGA monitor. The display output is shown in Figure 11. 8. The VGA port connections are shown in Figure 11. 9.

Demonstrate your working VGA monitor to respective RA. Attach the picture of your output screen.



Comments

**Observation/Comments on the obtained results/working of code.*

The green and red screen is exactly like the one that was expected.

Exercise

Code: Design module & testbench

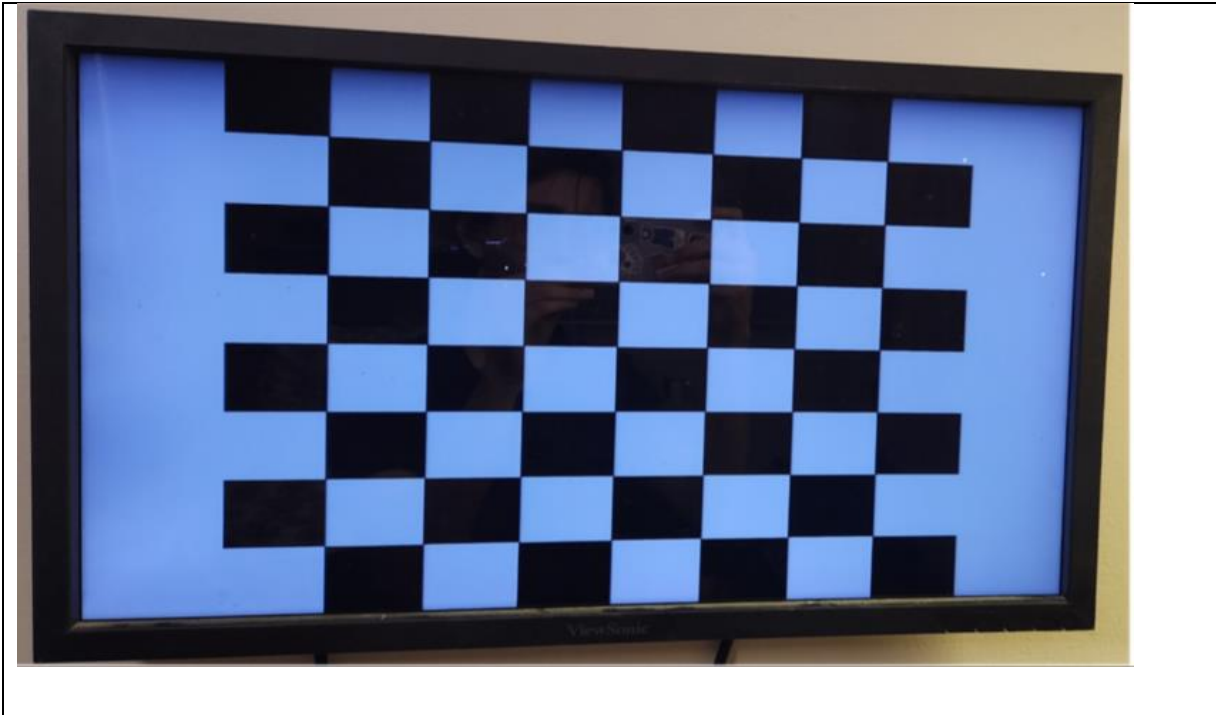
Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable naming.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

```
module PostLab_pixelgen(
    input clk_d,
    input [9:0] pixel_x,
    input [9:0] pixel_y,
    input video_on,
    output reg [3:0] red=0,
    output reg [3:0] green=0,
    output reg [3:0] blue=0
);
    parameter BOARD_SIZE = 480; // Board dimension (480x480)
    parameter GRID_SIZE = 8; // 8x8 board
    parameter SQUARE_SIZE = BOARD_SIZE / GRID_SIZE; // 60 pixels per square
    always @(posedge clk_d)
        begin
            if ((pixel_x < (640 - BOARD_SIZE) / 2) || (pixel_x >= (640 + BOARD_SIZE) / 2) ||
                (pixel_y < (480 - BOARD_SIZE) / 2) || (pixel_y >= (480 + BOARD_SIZE) / 2))
                begin
                    red <= 4'hF; green <= 4'hF; blue <= 4'hF; // White border around the board // White border around the board
                end
            else
                begin
                    if (video_on) begin
                        if (((pixel_x - (640 - BOARD_SIZE) / 2) / SQUARE_SIZE) % 2) ^
                            (((pixel_y - (480 - BOARD_SIZE) / 2) / SQUARE_SIZE) % 2)) begin
                            red <= 4'hF; green <= 4'hF; blue <= 4'hF; // White border around the board // White square
                        end else begin
                            red <= 4'h0; green <= 4'h0; blue <= 4'h0; // Black square
                        end
                    end else begin
                        red <= 4'h0; green <= 4'h0; blue <= 4'h0;
                    end
                end
            end
        end
endmodule
```

Results (Waveforms)

Attach a clear picture of the Monitor output here



Comments

**Observation/Comments on the obtained results/working of code.*

We modified the PixelGen code to display the checkerboard of size 480x480 with each box of size 60x60.



Assessment Rubric

Lab 11

VGA Controller

Name:	Student ID:
-------	-------------

Marks Distribution:

	Task No.	LR2 Code	LR5 Results
In - Lab	Task a	/20	/05
	Task b	-	-
	Task c	/10	/15
Exercise		/20	/15
Total: 100		/50	/35
CLO Mapping Learning Domain Level		CLO3	CLO3
		Psy-4	Psy-4

Affective Domain Rubric		Points	CLO Mapped	LDL
AR 7	Report Submission and Viva	/15	CLO 4	Aff-3

CLO	Total Points	Points Obtained
3	85	
4	15	
Total	100	

For description of different levels of the mapped rubrics, please refer to the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

**Lab Evaluation Assessment Rubrics**

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR1	Circuit Layout	Connections between circuit components are mostly wrong. Circuit layout is cluttered. Needs guidance to make correct equipment/ component connections.	Few of the connections made between circuit components are wrong. Circuit layout is not neat and clean as per standards.	Circuit layout and connections are correct but not all connections are neat and clean as per standards.	Neat, clean and correct connections of all the circuit components/ equipment are made as per standard circuit diagram.
LR3	Troubleshooting	Unable to identify the fault/minimal effort shown in troubleshooting.	Able to identify the fault but unable to remove it.	Able to identify the fault but partially removes it.	Able to identify the fault and takes necessary steps and actions to correct it.
LR5	Results & Plots	Figures/ graphs/ tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR10	Analysis	Results are not interpreted or are analyzed incorrectly.	Analysis of the obtained results is incomplete. Conclusions are not drawn.	Results are analyzed and concluded but are not well explained and justification is not provided with the help of reasoning.	Results are well analyzed supported with reasons. Good comparison is made between the theoretical and experimental results with correct and useful conclusion.
LR11	Design	Proposed design is unsubstantiated or does not satisfy most of given constraints. The breakdown of system into features is incomplete and still at lower resolution.	Justification for proposed design is weak, and it only satisfies some constraints. The breakdown of system is missing some features and resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies most given constraints. The breakdown of system into features seems complete, but resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies all given constraints. The breakdown of system into features seems complete and at appropriate resolution, given students' level.